



A Worst-Case Optimal Algorithm to Compute the Minkowski Sum of Convex Polytopes

Sandip Das¹, Subhadeep Ranjan Dev^{1(✉)}, and Swami Sarvottamananda²

¹ Indian Statistical Institute, Kolkata, Kolkata, India

sandipdas@isical.ac.in, info.subhadeep@gmail.com

² Ramakrishna Mission Vivekananda Educational and Research Institute, Howrah, Howrah, India

sarvottamananda@rkmvu.ac.in

Abstract. We propose algorithms to compute the Minkowski sum of a set of convex polytopes in $\mathbb{R}^d$. The input and output of the proposed algorithms are the face lattices of the input and output polytopes respectively. We first present the algorithm for the Minkowski sum of two convex polytopes. The time complexity of this algorithm is $O(d^\omega nm)$ where n and m are the face lattice sizes of the two input polytopes and ω is the matrix multiplication exponent ($\omega \sim 2.373$). Our algorithm for two summands is worst-case optimal for fixed d . We generalize this algorithm for r convex polytopes, say P_i , $1 \leq i \leq r$. The time complexity of this generalization is $O(\min\{d^\omega NM, d^\omega r \prod |P_i|\})$ where $N = \sum |P_i|$ is the total size of the face lattices of the r input polytopes and M is the size of the face lattice of their Minkowski sum $P_1 \oplus \dots \oplus P_r$. Our algorithm for multiple summands is worst-case optimal for fixed $d \geq 3$ and $r < d$.

1 Introduction

The Minkowski sum, initially defined by Hermann Minkowski [1864–1909], is an important concept, useful in computational geometry, computer graphics, robot motion planning, assembly planning, computer-aided design, computer-aided manufacturing and various other fields. The abundant literature from the late nineteenth century on the subject corroborates the importance and applicability of the concept. Recently Das et al. [10] implicitly used the Minkowski sums to compute the diameter, the width, the minimum enclosing/stabbing sphere, the maximum inscribed sphere and the minimum enclosing/stabbing cylinder for various types of inputs, some involving convex polygons/polytopes, for polyhedral distance functions [9]. The present algorithm was motivated by it.

The Minkowski Sum of two or more convex polytopes in $\mathbb{R}^d$ can be computed efficiently by a simple application of the optimal convex hull algorithm by Chazelle [6] or the output sensitive algorithm by Seidel [20]. The time complexities are $O(n^{\lfloor d/2 \rfloor})$ and $O(n^2 + h \log h)$ respectively for fixed d where n is the total number of Cartesian sums of the vertices of the input polytopes and

h is the size of the resultant Minkowski sum. For the sub-linear output size, an improved algorithm by Chan [5] can be used instead. The Minkowski sum computing algorithm using the results by Chazelle is a worst-case optimal algorithm, only when the summands and the sum, all are asymptotically worst-cases for the convex hulls as well as for the Minkowski sum. The Minkowski sum of two convex polygons in $\mathbb{R}^2$ can be computed optimally in linear time by a well known algorithm [4]. Bekker et al. [3] gave an optimal output sensitive algorithm, linear in the output size, to compute the Minkowski sum of two convex polyhedra in $\mathbb{R}^3$. In the case of $\mathbb{R}^d$, Fukuda et al. [14] presented an alternative non-optimal output sensitive algorithm for enumerating the faces in the Minkowski sum of convex polytopes. Their polynomial-time per-face algorithm did not improve upon the method using Seidel's $O(n^2 + h \log h)$ -time or Chan's sub-linear sized convex hull algorithm for output sensitive computation. Agarwal et al. [2], and later Fogel et al. [12], devised the algorithms to compute the Minkowski Sum of non-convex polygons in $\mathbb{R}^2$ and non-convex polyhedra in $\mathbb{R}^3$, respectively, using techniques of decomposition of non-convex polygons and polyhedra, respectively, by Chazelle et al. [7, 8].

In addition to the algorithmic results above, there are also several combinatorial results on the Minkowski sum of convex polytopes in $\mathbb{R}^d$. Grunbaum [16] proved that in the worst case, for $d \geq 3$, the number of faces of the Minkowski sum of two convex polytopes is the product of the number of faces of its two summands. This important fact contributes for the optimality of our proposed worst-case optimal algorithm. Adiprasito [1], Gritzmann et al. [15], Karavelas et al. [17], Weibel [21] and others derived various upper and lower bounds for the Minkowski sums.

The different approaches to compute the Minkowski sum of convex polytopes are to compute convex hulls [5, 6, 20], to compute the boundary faces of the resultant polytope using LPP [13, 14] or to use convolutions [3, 4, 19, 22]. The convolution method is well understood in the case of $\mathbb{R}^2$ and $\mathbb{R}^3$ but not in higher dimensions. In this paper, we compute the Minkowski sum of convex polytopes in $\mathbb{R}^d$ using their face lattices [5, 6, 11, 20], defined in Sect. 2, as input and output for traversal. Our traversal of the Minkowski sum is similar to the convolution method.

Our contributions in this paper are two worst-case optimal algorithms for fixed d to compute the face lattices of Minkowski sums for two and multiple convex polytopes, respectively, in $\mathbb{R}^d$. We are able to achieve worst-case optimal bounds in both the algorithms by devising efficient criteria to check the boundary faces of the Minkowski sum. For this check, we first characterize the faces and the incidences of the Minkowski sum in terms of the face lattice by providing algorithm specific necessary and sufficient conditions for faces (Lemma 1, Lemma 2 and Lemma 3) and for incidences (Lemma 6). A novel use of Lemma 4 is crucial to avoid duplicate faces. We use these characterizations to effectively remove the duplicate or invalid faces and incidences, on the one hand, and identify the valid faces and incidences of the Minkowski sum uniquely, on the other. Secondly, we augment the face lattices in a novel way. This allows us to apply

the sufficiency conditions in $O(d^\omega)$ time per candidate face of the Minkowski sum (Lemma 2(ii) and Lemma 3(ii)). The quantity ω , $\omega \sim 2.373$, in d^ω is the matrix multiplication exponent [18] and d^ω is the complexity of computing the rank of the $d \times 2d$ matrices in the two algorithms. The other significant points to note in this paper are an efficient face lattice traversal method independent of adjacency information and an effective table search for finding duplicates or identifying new faces (Sect. 4.2).

A brief summary of time complexities in our algorithms is as follows. Let P and Q be two convex polytopes in $\mathbb{R}^d$ with face lattice sizes n and m , respectively. Our algorithm for two summands computes the face lattice of the Minkowski sum $P \oplus Q$ of P and Q in $O(d^\omega nm)$ time (Theorem 2). This running time is worst-case optimal for any fixed $d \geq 3$. For $d \leq 2$, a trivial modification to our algorithm suffices to compute the face lattice of $P \oplus Q$ in $O(n + m)$ time. Furthermore, let $P_1, P_2, \dots, P_r$ be r convex polytopes in $\mathbb{R}^d$ with respective face lattice sizes $n_1, n_2, \dots, n_r$. Let $N = \sum_{i=1}^r n_i$ and let M denote the size of the face lattice of the Minkowski sum, $\bigoplus_{i=1}^r P_i$, of the r polytopes. We generalize our previous algorithm to compute the face lattice of the Minkowski sum $\bigoplus_{i=1}^r P_i$ of r summands in time $O(\min\{d^\omega NM, d^\omega r \prod_{i=1}^r n_i\})$ (Theorem 3). This running time is optimal for fixed d when $r < d$ and is our primary contribution in this paper. Note that the size of the Minkowski sum for $r \geq d$ is bounded by the sum of sizes of $(d - 1)$ -subset Minkowski sum among r polytopes.

The remaining paper is organized as follows. In Sect. 2 we provide a few preliminary definitions and notations used throughout the paper. In Sect. 3 we describe an augmentation to the input face lattices and develop some basic methods needed by our algorithms. Next, in Sect. 4 we present our first algorithm to compute the Minkowski sum of two convex polytopes in $\mathbb{R}^d$. In the same section, we also present various geometric results on the face lattice of convex polytopes and their Minkowski sum that prove the correctness and complexity of our algorithm. In Sect. 5 we generalize the method used in our first algorithm to efficiently compute the Minkowski sum of multiple convex polytopes in $\mathbb{R}^d$.

2 Preliminaries

Let S_1 and S_2 be two sets of points in $\mathbb{R}^d$. The *Minkowski sum* of these two sets, denoted by $S_1 \oplus S_2$, is defined as the set $\{p \mid p = p_1 + p_2, \forall p_1 \in S_1 \text{ and } \forall p_2 \in S_2\}$ where $p_1 + p_2$ denotes the sum of the positional vectors of p_1 and p_2 . This definition is equally applicable to the Minkowski sum of convex polytopes or, as a matter of fact, to any geometric objects as they can be equivalently seen as sets of points. We assume, for the sake of simplicity, that the origin of reference lies strictly in the interior of all the input convex polytopes, i.e., not on the boundary or in the exterior. However, note that the illustrations in the paper, for the sake of clarity, are in general positions.

It is a well-known fact that the Minkowski sum of two convex sets is also convex. Extending this to polytopes we have the following observation (see Fig. 1 for an example).

Observation 1. The Minkowski sum of any two convex polytopes is also a convex polytope.

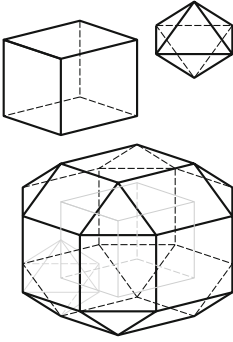


Fig. 1. Minkowski sum of two convex polytopes in $\mathbb{R}^3$.

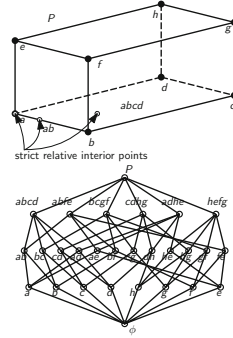


Fig. 2. Face lattice $\mathcal{L}(P)$ as a Hasse diagram of the cuboid P . For example, $ab \prec abcd$ as well as $ab \succ abcd$.

Let S be a set of points in $\mathbb{R}^d$. The *affine space* of S , denoted by $\text{affine}(S)$, is the affine combinations of the points in S , i.e., the set $\{p \mid p = \sum_{i=1}^k \lambda_i p_i, \forall i, 1 \leq i \leq k, p_i \in S, \lambda_i \in \mathbb{R} \text{ and } \sum_{i=1}^k \lambda_i = 1\}$. Every affine space A has an associated *affine dimension* which is the cardinality of its basis and is denoted by $\dim(A)$. In our algorithms we represent an affine space A by a point in A and any of its several bases consisting of $\dim(A)$ number of basis vectors.

We represent a *hyperplane* h in $\mathbb{R}^d$ by an equation of the form $\vec{x} \cdot \vec{n} = c$, where $\vec{n}$ is the normal to h and c is a fixed constant. The two open halfspaces determined by h are $\vec{x} \cdot \vec{n} > c$ and $\vec{x} \cdot \vec{n} < c$ and are denoted by h^+ and h^- respectively. We say that a hyperplane h is a *supporting hyperplane* of a polytope P if $P \cap h \neq \emptyset$ and $P \cap h^+ = \emptyset$, i.e., P touches h and its interior lies completely in h^- because of our assumption that origin lies in the interior.

A *nontrivial face* of a polytope P is the intersection of P with a supporting hyperplane h of P . The only two *trivial faces* of P are the face P itself and the null face ϕ . Observe that the faces of any convex polytope are also convex. Moreover, for a face f and a supporting hyperplane h of P if $f = P \cap h$ we call h the *supporting hyperplane* of P on f . We have the following observation.

Observation 2. Every supporting hyperplane h of a d dimensional polytope P corresponds to exactly one face of P and every nontrivial face f of P has one supporting hyperplane if f is a $d - 1$ dimensional face or infinite supporting hyperplanes otherwise.

We call a $(k - 1)$ -dimensional face, say f , of a k -dimensional face F a *facet* of F . We denote this by $f \prec F$ and $F \succ f$. A face f is called a *subface* of a face

F and F a *superface* of f if there exists a sequence of faces $f_1, f_2, \dots, f_i$ (i can be 0) such that $f \prec f_1 \prec f_2 \cdots \prec f_i \prec F$.

Every face f of P has a corresponding affine space $\text{affine}(f)$ and a corresponding affine dimension $\dim(f)$. For brevity, we use the same terms that we used for point sets earlier. In subsequent sections, we represent an affine space A , with $\dim(A) = k$, by the tuple $\langle \vec{p}, \vec{u}_1, \vec{u}_2, \dots, \vec{u}_k \rangle$, if the affine space A corresponds to the parametric equation $\vec{x}(\vec{t}) = \vec{p} + \sum_{i=1}^k t_i \cdot \vec{u}_i$ where $\vec{p}$ is the vector representation of a point in A , the set $\{\vec{u}_1, \vec{u}_2, \dots, \vec{u}_k\}$ is a vector basis of A , the vector $\vec{t} = (t_1, t_2, \dots, t_k)$ is the parameter with $\vec{t} \in \mathbb{R}^k$ and $\vec{x}(\vec{t})$'s are points of the affine space A .

We assume that the input consists of the face lattices of summand convex polytopes and either $\mathcal{H}$ -representations or the $\mathcal{V}$ -representations of the summands. We briefly describe the face lattice of any convex polytopes and a linear-time augmentation of any face lattice, that we need for the algorithm, in the next section.

3 Augmented Face Lattice of Convex Polytopes

3.1 Face Lattice of a Convex Polytope

Let P be a convex polytope of affine dimension δ , where $\delta \leq d$, i.e. the polytope P may be in a subspace of $\mathbb{R}^d$. We first define the face lattice [5, 6, 11, 20] of any convex polytope for reference.

Definition 1. *The face lattice, denoted by $\mathcal{L}(P)$, of a convex polytope P is a Hasse diagram where the nodes are faces of P including a unique null face ϕ and the polytope P . Moreover, the reflexive, anti-symmetric and transitive binary relation $\prec^*$ or, equivalently, $\succ^*$, the transitive closure of $\prec$ or $\succ$, respectively, is used as the partial order relation for the Hasse diagram.*

For our purpose, we represent the Hasse diagram of the face lattice as a layered graph in the algorithm, with the nodes arranged in layers according to the dimension k , $-1 \leq k \leq \delta$, of their faces. See Fig. 2 for the face lattice of a cuboid in $\mathbb{R}^3$. Though we need additional description of the faces in the nodes in many applications, in this paper, we only need either the $\mathcal{H}$ -representation of the polytope stored in $(\delta - 1)$ -dimensional faces, i.e. facets, or the $\mathcal{V}$ -representation of the polytope stored in 0-dimensional faces, i.e. vertices. Any representation that allows us to augment the face lattice described below is acceptable. We briefly describe the data structure representing $\mathcal{L}(P)$ below.

3.2 Data Structure for a Face Lattice

For convenience, we denote the set of all faces of the polytope P by $\mathcal{F}(P)$ and its subsets consisting of faces of dimension k by $\mathcal{F}_k(P)$, $-1 \leq k \leq \delta$. The face lattice of P has nodes arranged in $(\delta + 2)$ layers where the nodes in the k -th layer represent the faces in $\mathcal{F}_k(P)$. First, we denote the face corresponding to a node x

by $face(x)$, and, vice versa, the node corresponding to a face f by $node(f)$. In this paper, due to one-to-one mapping, the nodes and their corresponding faces refer to each other uniquely and we use them interchangeably in the discussions. Next, for the sake of brevity, $dim(x)$ and $affine(x)$ for a node x denote the dimension and affine space respectively of the face $face(x)$. We store $dim(x)$ and the tuple representing $affine(x)$ in each node x , except in the node of the null face ϕ , for which $dim(x) = -1$. Each node x also maintains two lists, $super(x)$ and $sub(x)$ where the list $super(x)$ is the list of all nodes x' such that $face(x') \succ face(x)$ and $sub(x)$ is the list of all nodes x' such that $face(x') \prec face(x)$. We also store a strict relatively interior point of the face of the node, denoted by $point(x)$, in each node x , except the node of null face ϕ . By *relative interior*, we mean that the point is an interior, i.e. non-boundary, point of face $face(x)$ relative to its affine space and by *strict*, we mean that the point is not a relative interior point of any of the proper subfaces of $face(x)$. See Fig. 2 for illustration of a strict relatively interior point. In the discussion below, we show how we augment the face lattice $\mathcal{L}(P)$ to include $point(x)$ and $affine(x)$ in each node x , in time linear in the size of the face lattice. We assume that the face lattice inherently has $dim(x)$, $sup(x)$ and $super(x)$ stored in its each node x .

3.3 Augmenting a Face Lattice

In literature, the $\mathcal{V}$ -representation of the convex polytope P is the minimal set of points whose convex hull is the polytope P . An alternative representation is the $\mathcal{H}$ -representation of P which is the set of minimal halfspaces whose intersection is the polytope P . Both the representations equivalently and uniquely describe the polytope P for our purpose. The conversion from one representation to another requires a simple top-down or bottom-up traversal of the face lattice. We can do this in linear time. Thus, we can augment the face lattice of the polytope P by the method given below, if the input contains either the $\mathcal{H}$ -representation or the $\mathcal{V}$ -representation of the polytope P .

In the following discussion, we assume that the $\mathcal{V}$ -representation of the convex polytope P , equivalently the coordinates of all the vertices in $\mathcal{F}_0(P)$, is provided. We do a bottom-up traversal on $\mathcal{L}(P)$ to augment the face lattice with $point(x)$ for each node x . Initially, for each node x corresponding to vertices, i.e. the faces in $\mathcal{F}_0(P)$, we set $point(x)$ to be the coordinates of the vertex determined by the $\mathcal{V}$ -representation. Then at intermediate steps, at each node x with $dim(x) > 0$ we arbitrarily select two nodes y and y' in the list $sub(x)$ and set $point(x)$ to be the middle point $(point(y) + point(y'))/2$. We can easily show that $point(x)$ is a strict relative interior point of face $face(x)$. It is interior because of convexity, relative because of being an affine combination of points in subfaces and strict because the affine spaces of distinct subfaces are distinct.

Next, we show how we augment in linear time the face lattice to include the affine spaces of all the nodes required by our algorithm.

As mentioned earlier the parametric equation of the affine space of a k -dimensional face f is represented by the $(k+1)$ tuple $\langle \vec{p}, \vec{u}_1, \vec{u}_2, \dots, \vec{u}_k \rangle$. In this representation, $\vec{p}$ is the vector representing a strict relative interior point of f and

the set $\{\vec{u}_1, \vec{u}_2, \dots, \vec{u}_k\}$ is a basis of $\text{affine}(f)$. We show below how we compute the tuple $\langle \vec{p}, \vec{u}_1, \vec{u}_2, \dots, \vec{u}_k \rangle$ representing the affine space $\text{affine}(x)$ for any node x in $\mathcal{L}(P)$. We first make the following observation.

Observation 3. If f and g are two faces of P such that $f \succ g$ then any point in the strict relative interior of the face f is *affinely independent* of the points in the face g , i.e., no strict relative interior point of the face f is an affine combination of points in the face g .

In short, the observation above is the consequence of the fact that the affine dimension increases as we add the strict relative interior points of the superfaces. Thus, in the pseudocode below, we compute the affine space of each of the subfaces of $\text{face}(x)$ in $\mathcal{L}(P)$ recursively, in linear time.

```

procedure AUGMENTAFFINE( $x$ )
  if  $\text{affine}(x)$  is unset then
    if  $x$  is a vertex then
      | Set  $\text{affine}(x)$  for the vertex appropriately;
    else
      foreach  $y \in \text{sub}(x)$  do
        | AUGMENTAFFINE( $y$ )
      Choose a node  $y$  arbitrarily from  $\text{sub}(x)$ ;
      Set  $\text{affine}(x)$  as  $\text{affine}(y)$  with additional basis vector
       $\text{point}(y) - \text{point}(x)$ ;

```

The correctness of the recursive algorithm follows from the observation above. We, therefore, have the following theorem.

Theorem 1. *We can augment the face lattice $\mathcal{L}(P)$ of the convex polytope P with affine dimensions, affine spaces and strict relative interior points in linear time for either the $\mathcal{V}$ -representation or the $\mathcal{H}$ -representation of the polytope P . The space required is $O(d^2 \cdot |\mathcal{L}(P)|)$.*

Once we compute the affine spaces of the faces of the summand convex polytopes, we can readily compute the affine dimension and the affine space of the Minkowski sum of the faces, which may be less than the sum of individual dimensions of the summand faces. This is required to determine if the sum is a valid face and then, if it is, to determine the appropriate layer of the face lattice of the sum. We show next how we compute the affine dimension.

3.4 Determination of the Dimension of the Minkowski Sum of Faces

Let P and Q be two convex polytopes with face lattices $\mathcal{L}(P)$ and $\mathcal{L}(Q)$. Let x be a node in $\mathcal{L}(P)$ and x' a node in $\mathcal{L}(Q)$. We show in pseudocode below, how we determine the affine dimension of $\text{face}(x) \oplus \text{face}(x')$. We note that $\text{face}(x) \oplus \text{face}(x')$ may not be a valid face of $P \oplus Q$. It may be a partial face or it may partially lie inside P .

We also note that for any two faces f in P and g in Q the affine space $\text{affine}(f \oplus g) = \text{affine}(f) \oplus \text{affine}(g)$. This follows from the definition of the affine spaces and the Minkowski sum. So, briefly, we consider the combined set of affinely independent vectors in $\text{affine}(x)$ and $\text{affine}(x')$ and return the resulting rank as the dimension of $f \oplus g$. The function $\text{RANK}()$ in our algorithm computes the rank of a matrix using the fastest rank determination algorithm. We give below the pseudocode to determine the dimension of $f \oplus g$ where $f = \text{face}(x)$ and $g = \text{face}(x')$ in $\mathcal{L}(P)$ and $\mathcal{L}(Q)$ respectively.

```

function DIM( $x, x'$ )
     $\langle \vec{p}, \vec{u}_1, \vec{u}_2, \dots, \vec{u}_k \rangle \leftarrow \text{affine}(x);$ 
     $\langle \vec{p}', \vec{u}'_1, \vec{u}'_2, \dots, \vec{u}'_{k'} \rangle \leftarrow \text{affine}(x');$ 
     $r \leftarrow \text{RANK}[\vec{u}_1 \ \vec{u}_2 \ \dots \ \vec{u}_k \ \vec{u}'_1 \ \vec{u}'_2 \ \dots \ \vec{u}'_{k'}];$ 
    return  $r$ ;

```

The runtime complexity of the function DIM is d^ω where ω is the matrix multiplication exponent [18]. We will use this time complexity later. The correctness of the function above follows from the fact that $\text{affine}(A \oplus B) = \text{affine}(A) \oplus \text{affine}(B)$ for any two sets of points A and B . We use the methods described above in the subsequent sections to compute the Minkowski sum of convex polytopes.

4 The Minkowski Sum of Two Convex Polytopes

Let P and Q be two convex polytopes in $\mathbb{R}^d$ with the face lattice representations $\mathcal{L}(P)$ and $\mathcal{L}(Q)$. The face lattice sizes are n and m respectively. In this section, we present an $O(d^\omega nm)$ -time and $O(d^2 nm)$ -space algorithm to compute the Minkowski sum $P \oplus Q$. In Sect. 4.1 we first present some important characterizations for the Minkowski sum $P \oplus Q$ necessary for our algorithm. We then present the main algorithm of the algorithm in Sect. 4.2.

In this section, we assume unless mentioned otherwise, that the affine dimension of the polytope $P \oplus Q$, i.e., $\dim(P \oplus Q)$, is d . Otherwise, if $\dim(P \oplus Q) < d$, we compute the Minkowski sum in lower affine dimension by projecting the input to the affine space of the polytope $P \oplus Q$.

4.1 Necessary and Sufficient Conditions for the Faces of $P \oplus Q$

First we provide a necessary condition for the faces of the polytope $P \oplus Q$. The following lemma is a well known fact so the proof is omitted.

Lemma 1 (Necessary Condition for a face F in $P \oplus Q$). *Let P and Q be convex polytopes. If F is any face of $P \oplus Q$ then there exist a unique face f in P and a unique face g in Q such that $F = f \oplus g$.*

As a consequence of Lemma 1, all the faces of the polytope $P \oplus Q$ are of the form $f \oplus g$ where f is a face of the polytope P and g is a face of the polytope Q . However, not all possibilities of $f \oplus g$ are faces of the polytope $P \oplus Q$. To remove the invalid possibilities we need sufficient conditions. By definition of a face, for $f \oplus g$ to be a face of $P \oplus Q$, a sufficient condition is that there must be a supporting hyperplane h of $f \oplus g$. We state the condition below.

Observation 4 (Sufficient condition). Let f and g be faces of convex polytopes P and Q respectively. The Minkowski sum $f \oplus g$ is a face of polytope $P \oplus Q$, if there exists a supporting hyperplane h such that $(P \oplus Q) \cap h^+ = \emptyset$ and $(P \oplus Q) \cap h = f \oplus g$.

This condition, however, can not be efficiently checked. So it is not practical. We need a computationally efficient sufficient condition. Therefore we present alternative conditions. First, we suggest a sufficient condition for $f \oplus g$ to be a facet of polytope $P \oplus Q$ and then we generalize the condition for other lower dimensional faces. For facets of $P \oplus Q$, the condition is simplified because there is only one choice of supporting hyperplane h , $h = \text{affine}(f \oplus g)$, since the affine dimension will be $d - 1$ because of our assumption, i.e., the affine space will be the supporting hyperplane because it happens to be the only choice. Now, the hyperplane h will be a supporting hyperplane of $f \oplus g$, if, equivalently (1) every point of the polytope $P \oplus Q$, other than of $f \oplus g$, is in h^- , (2) every vertex in the polytope $P \oplus Q$, other than the vertices of $f \oplus g$, is in h^- , since due to convexity every point in $P \oplus Q$ is a convex combination of its vertices, or (3) every strictly relative interior point of possible $f_i \oplus g$ and $f \oplus g_j$ are in h^- where f_i 's and g_j 's are the immediate superfaces of f and g respectively. The criterion in (3) can be computed efficiently and the equivalence of the three criteria can be proved. Summarily, the idea of the proof is that if $f \oplus g$ is not a facet then at least one of the several $f \oplus g_j$'s and $f_i \oplus g$'s will have its interior outside h^- . We present the sufficient condition of (3) in the following lemma. We omit the proof because of space constraint.

Lemma 2 (Sufficient condition for a facet $f \oplus g$ in $P \oplus Q$). Let f be a face of a convex polytope P and g be a face of a convex polytope Q . The Minkowski sum $f \oplus g$ is a facet of polytope $P \oplus Q$, if

- (i) $\dim(f \oplus g) = d - 1$, and
- (ii) $\forall f', f' \succ f$, $\text{point}(f') + \text{point}(g) \in h^-$ and $\forall g', g' \succ g$, $\text{point}(f) + \text{point}(g') \in h^-$ where h is $\text{affine}(f \oplus g)$.

Once we have a sufficient condition for the facets, we can get sufficient conditions for the other faces by treating them as the facets of any of their immediate superfaces. We can effectively and efficiently do this by checking the conditions relative to the affine space of the immediate superfaces. So, if we are able to compute all the facets of $P \oplus Q$ correctly, then we can also compute the $(d - 2)$ -faces of the polytope $P \oplus Q$ by checking the sufficient conditions, then we can continue to $(d - 3)$ -faces, and so on. This will allow us to check sufficient conditions for

the faces of all affine dimensions. We show later how we do this efficiently. We present the generalized sufficient condition in the lemma below. As mentioned earlier, the face $f \oplus g$, in the lemma must be part of a known face of $P \oplus Q$ to be its facet. The proof, which is similar to that of Lemma 2 is omitted.

Lemma 3 (Sufficient condition for a face $f \oplus g$ in $P \oplus Q$). *Let $F \oplus G$ be a face of polytope $P \oplus Q$ where F and G are faces of convex polytopes P and Q respectively. Let f and g be any faces of P and Q , such that $f \prec F$ and $g \prec G$ respectively and let $\dim(f \oplus g) = \dim(F \oplus G) - 1 = k$. Then $f \oplus g$ is a facet of $F \oplus G$, and hence is a face of $P \oplus Q$, if,*

- (i) $\dim(f \oplus g) = k$, and
- (ii) $\forall f', F \succ \dots \succ f' \succ f$, $\text{point}(f') + \text{point}(g) \in A^-$ and $\forall g', G \succ \dots \succ g' \succ g$, $\text{point}(f) + \text{point}(g') \in A^-$ where A is the hyperplane, $A = \text{affine}(f \oplus g)$, in the relative subspace $\text{affine}(F \oplus G)$, i.e., in $\mathbb{R}^{k+1}$.

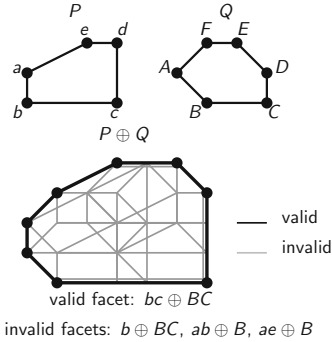


Fig. 3. Invalid sums $f \oplus g$ can be partial facets, partially on the boundary or completely in the interior.

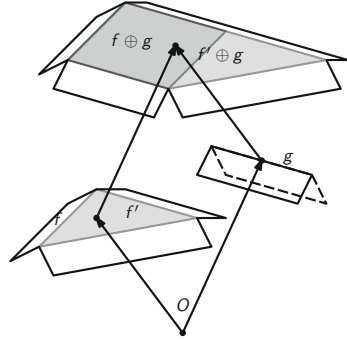


Fig. 4. Invalid partial facet $f \oplus g$ not satisfying the sufficiency condition, i.e. $\text{point}(f' \oplus g) \notin h^-$.

The necessity and sufficiency lemmas presented above, unfortunately, do not remove the possibility that same face of the polytope $P \oplus Q$ is computed multiple times. We need an efficient method to identify duplicates. Even after identifying duplicates correctly, we also need an efficient method to add missing incidences at a later stage correctly. First we note, owing to Lemma 1, that multiple $f_i \oplus g_i$'s can not give the same face of $P \oplus Q$. Second, if $f_i \oplus g_i$'s refer to partial faces on the boundary of $P \oplus Q$ with same affine space then the suprema of f_i 's and g_i 's will be faces F and G respectively, such that $F \oplus G$ is a face of $P \oplus Q$ with the same affine space. This allows us to add a node corresponding only to the Minkowski sum of suprema, $F \oplus G$, when we do a top down computation. We present this property in the following lemma needed in the algorithm to halt the top down processing (Figs. 3 and 4).

Lemma 4. *Let f be a face of P and g be a face of Q such that $\text{affine}(f \oplus g) = A$. Let h be a supporting hyperplane on A , i.e., $(P \oplus Q) \cap h \subseteq A$, and $(P \oplus Q) \cap h^+ = \emptyset$. Then there exist suprema faces F and G of P and Q respectively, such that $f \prec \dots \prec F$, $g \prec \dots \prec G$ respectively, $\text{affine}(F \oplus G) = A$ and $F \oplus G$ is a face of $P \oplus Q$.*

The proof of Lemma 4 is omitted. An immediate consequence of the lemma is all $f_i \oplus g_i$'s that account for partial faces will have a proper face $F \oplus G$ which is unique and therefore in a top down computation, once we compute a proper face of $P \oplus Q$, we neither encounter any partial faces nor we compute the same face twice. This is the consequence of the following lemma which we state without the proof.

Lemma 5. *Let f and f' be two faces of P and g and g' be two faces of Q . If $f \oplus g$ and $f' \oplus g'$ correspond to the same face of $P \oplus Q$ then $f \equiv f'$ and $g \equiv g'$.*

Consequently, if we compute a suprema face of two summands, we are sure that there are no other summands for the same suprema face. Thus there would be no partial faces, no invalid faces and no duplicate faces. The only task that remains in the present section is the characterization of incidences of the computed faces. The incidences of the facets to P computed by the application of Lemma 2 are obvious. Following Lemma 1 and Lemma 3, we only need to implement the remaining incidences by the application necessary and sufficient condition for the facets of the faces of the Minkowski sum $P \oplus Q$ in the algorithm. We present the necessary and sufficient condition for these incidences in the following lemma.

Lemma 6 (Necessary and Sufficient Condition for an Incidence $f \oplus f' \prec g \oplus g'$ in $P \oplus Q$). *Let f and g be faces of P and f' and g' be faces of Q where P and Q are convex polytopes. Then $f \oplus f' \prec g \oplus g'$ iff*

- (i) $f \oplus f'$ and $g \oplus g'$ are faces of $P \oplus Q$,
- (ii) $\dim(f \oplus f') = \dim(g \oplus g') - 1$, and
- (iii) $f \prec \dots \prec g$ and $f' \prec \dots \prec g'$

The above lemma can be proved by analyzing the supporting hyperplanes of faces $f \oplus f'$ and $g \oplus g'$. The immediate consequence of this lemma is that whenever we compute a facet of a face of $P \oplus Q$ we can immediately add an incidence relation whether it is newly computed or not. The lemma also ensures that we do not miss any incidences during the execution of the algorithm.

We describe the algorithm to compute the Minkowski sum in the next section.

4.2 The Minkowski Sum Algorithm

To simplify the discussion we treat the nodes in the face lattices as the corresponding faces. Our algorithm computes the faces of $P \oplus Q$ and the incidences layer by layer in a top-down manner. At each layer k , $-1 \leq k \leq d$, we maintain a list of the computed nodes in $\mathcal{F}_k(P \oplus Q)$. For each computed node z in

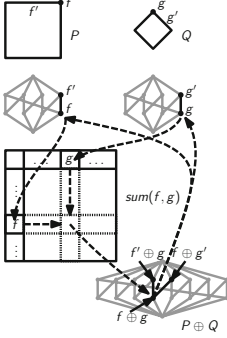


Fig. 5. Identifying replication using the table *sum*.

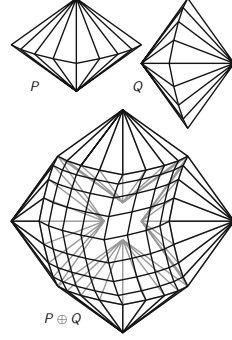


Fig. 6. Worst case: Even in $\mathbb{R}^3$, the number of faces, i.e., sum of vertices, edges and facets, in $P \oplus Q$ can be $\Theta(nm)$.

$\mathcal{F}(P \oplus Q)$, we store its summand nodes x and y from $\mathcal{L}(P)$ and $\mathcal{L}(Q)$ respectively. We create a 2-dimensional table *sum* that takes as indices two nodes x in $\mathcal{L}(P)$ and y in $\mathcal{L}(Q)$ such that the table entry maps to the node z . Other table entries will be empty. This table is updated whenever a new node is added to $\mathcal{L}(P \oplus Q)$. See Fig. 5. Internally, we create unique consecutive indices for the nodes.

We describe the computation of $P \oplus Q$ below. Initially, the set $\mathcal{F}_d(P \oplus Q)$ contains a single node representing the trivial face $P \oplus Q$. We compute the set $\mathcal{F}_k(P \oplus Q)$ inductively from the already computed set $\mathcal{F}_{k+1}(P \oplus Q)$, from $k = d - 1$ down to 0, as follows. For each node $z \in \mathcal{F}_{k+1}(P \oplus Q)$ we look for the facets of z among the candidate Minkowski sums involving the subfaces x_i 's and y_j 's of the summands x and y of the sum z . A sum $x_i \oplus y_j$ is a candidate sum if $\dim(x_i \oplus y_j)$ is $\dim(z) - 1$. To verify whether a candidate sum, i.e $x_i \oplus y_j$, is a new valid facet of z we need to check two conditions: (1) whether the sum satisfies Lemma 3 and (2) a node corresponding to the sum does not already exist in $\mathcal{L}(P \oplus Q)$. Condition (1) depends only on the immediate superfaces of x_i and y_j and the affine space of z . This can be readily checked since we have all necessary information with us from the previous iteration. Condition (2) can be verified by checking the table *sum*(x, y). Once the sum of $x_i \oplus y_j$ is determined to be a valid immediate subface of z we create a node z' in $\mathcal{L}(P \oplus Q)$ corresponding to the sum of x_i and y_j along with proper incidences and update the table *sum* as per Lemma 6. During traversal if we reach either a $k - 1$ dimensional candidate node or a non-facet k dimensional candidate node we do not traverse to further subfaces of x' and y' as outlined by Lemma 4. This concludes our algorithm. We have the following theorem as a result.

Theorem 2. *The Minkowski sum $P \oplus Q$ of the convex polytopes P and Q in $\mathbb{R}^d$ of face lattice sizes n and m respectively can be computed in $O(d^\omega nm)$ -time*

and $O(d^2nm)$ -space. Furthermore, the algorithm is worst-case optimal in n and m , i.e., for fixed d , when $|P \oplus Q| = O(nm)$.

Proof. Correctness: A direct consequence of Lemma 1 is that in order to compute the faces of $P \oplus Q$ we need only check the faces of type $f \oplus g$ where f is a face of P and g a face of Q . Not all such faces are valid and Lemma 3 ensures that we choose only the correct faces of $P \oplus Q$. Furthermore Lemma 4 and 5 guarantee that no duplicate nodes are generated by our algorithm.

Time Complexity: The run-time complexity is $O(d^\omega nm)$ because our algorithm traverses each pair of incidences of the face lattices $\mathcal{L}(P)$ and $\mathcal{L}(Q)$ only once. The expression d^ω is due to the time required to compute rank in the necessary and sufficient condition.

Space Complexity: The $O(d^2nm)$ space requirement is due to the extra storage for face-lattice augmentation when we store d -dimensional vectors in the nodes.

Optimality: We assume that d is fixed in the following discussion. We show that the worst case space and time complexities are tight. See Fig. 6 for a worst-case construction in $\mathbb{R}^3$. For $d > 3$, we describe a d -dimensional construction in Sect. 4.4. Since there are $\Theta(nm)$ vertices in the Minkowski Sum in the construction, any reasonable representation of the polytope, $\mathcal{V}$ -representation, $\mathcal{H}$ -representation, face enumeration, facet enumeration, facial structure, face lattice, etc.; all will have $\Omega(nm)$ length. This is a lower bound for any Minkowski Sum algorithm for these constructions because no algorithm can report the vertices in less time. So the complexity of any algorithm that constructs the Minkowski sum for this example will be $\Omega(nm)$. Since our algorithm runs in $\Theta(nm)$, i.e., linear in output length, therefore the lower bound is tight and our algorithm is worst-case optimal. $\square$

4.3 Minkowski Sum from Other Input Representations

V-representation or H-representation. We note that instead of the face lattices of the convex polytopes P and Q , if only the $\mathcal{V}$ -representations or $\mathcal{H}$ -representations of P and Q are provided, then we can compute the Minkowski Sum $P \oplus Q$ by first computing the face lattices of P and Q using the best known algorithm for convex hull construction before applying our techniques.

This may lead to a better performing algorithm than existing techniques in many cases except, for example, when Chan's and Seidel's output sensitive convex hull applications perform better. For example, if we use the best of Chazelle's, Seidel's or Chan's algorithm to compute the convex hull, the time complexity of our algorithm would be $O(nm + T(\mathcal{L}(P)) + T(\mathcal{L}(Q)))$ instead of the current $O(T(\mathcal{L}(P \oplus Q)))$ for fixed d where $T(\mathcal{L}) = \min\{T_{bc}(\mathcal{L}), T_{rs}(\mathcal{L}), T_{tc}(\mathcal{L})\}$ and T_{bc} , T_{rs} and T_{tc} are the time complexities of the convex hull algorithms of Chazelle, Seidel and Chan, respectively, to compute the final convex hull face lattice $\mathcal{L}$. For illustration, this is always better than Chazelle's convex hull application and our $O(nm + (n + m) \log(n + m))$ -time, surprisingly, fares better compared to Seidel's output sensitive $O((\dots)^2 + h \log h)$ whenever $h \log h$, $h = |P \oplus Q|$, is $\omega(nm)$. $\square$

Face Enumeration Suppose we are provided an enumeration of all the faces (or only facets and vertices) of the input Polytopes P and Q . In this case, there are algorithms in the literature to compute face lattices from the face enumerations in $O(F \log F)$ time using a dictionary [5, 20], where $F = |\mathcal{F}|$, $\mathcal{F}$ is the set of faces of the input convex polytope. We expect that we should have at least the affine space of each face in the input, or a way to compute it efficiently, in order to compute the incidences of the face lattice. For the case of enumeration of only facets and vertices, we need more data on facets and vertices. Thus if the faces of input polytopes P and Q are enumerated instead of face lattice as input, then the resulting complexity of our algorithm which computes the face lattice from the face enumeration first will be $O(nm + (n + m) \log(n + m))$ -time in the worst case. $\square$

From the above discussion, it can be seen that our algorithm fares quite well even in the case when the input is provided in common representations. Summarily, only a term $(n + m) \log(n + m)$ is added (which is the time complexity for sort). In the next section, we give an example to prove the worst-case optimality of the algorithm.

4.4 An Example to Prove Worst-Case Optimality

Let us consider the convex polytopes (in fact, polygons placed orthogonally) P_i , $1 \leq i \leq d - 1$ in d -dim that is the convex hull of n_i points with the coordinates $(x_1 = 0, \dots, x_{i-1} = 0, x_i = h(1 - t^2), x_{i+1} = 0, \dots, x_{d-1} = 0, x_d = t)$ for n_i distinct values of t , $-1 \leq t \leq 1$. In short, except for coordinates x_i and x_d , other coordinates are 0. We construct convex polytopes $P = P_1 \oplus \dots \oplus P_k$ and $Q = P_{k+1} \oplus \dots \oplus P_{d-1}$. Note that the origin is on the bases of polytopes P and Q . In this section, we consider d as fixed and express the complexities independent of d .

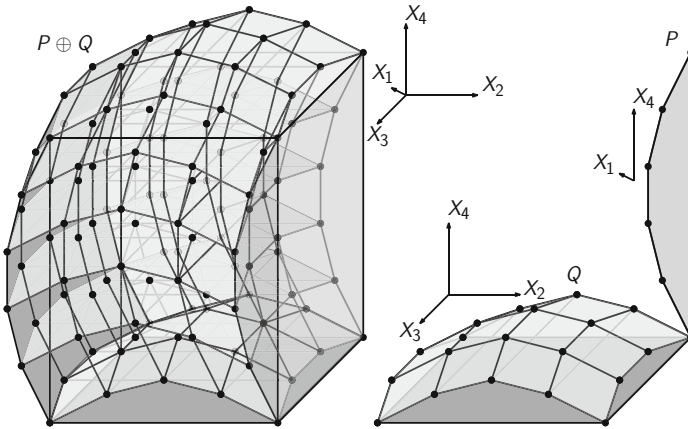


Fig. 7. Construction of the worst-case polytope $P \oplus Q$ in $\mathbb{R}^4$ to prove the optimality

We draw the above example in $\mathbb{R}^4$, where $P = P_1$ and $Q = P_2 \oplus P_3$. See Fig. 7 for illustration. If we wish to construct d -dimensional input polytopes instead of k or $d - k - 1$ dimensional polytopes then we may consider the polytopes $P \oplus \epsilon$ -cube and $Q \oplus \epsilon$ -cube instead, where the ϵ -cube is an axis-parallel d -dimensional hypercube of ϵ size.

The number of vertices, as well as number of faces and the size of face-lattice of P is $\Theta(n_1 n_2 \dots n_k)$. Let the size of the face lattice be n . The number of vertices, as well as number of faces and the size of face-lattice of Q is $\Theta(n_{k+1} n_{k+2} \dots n_{d-1})$. Let the size of face lattice be m . The number of vertices, as well as number of faces and the size of face-lattice of $P \oplus Q$ is $\Theta(n_1 n_2 \dots n_{d-1})$. This is $\Theta(nm)$, since every point $u \oplus v$, where u is a vertex of P and v is a vertex of Q , is a vertex of $P \oplus Q$ (i.e. on the boundary of the convex hull).

Our algorithm computes the Minkowski sum for this example in $\Theta(nm)$, whereas the best known output sensitive algorithm, i.e. Seidel's, is $\Theta((nm)^2 + h \log h)$. Since the output size is $\Theta(nm)$ for this worst-case, our algorithm is linear in the output size and therefore optimal. This completes the argument for worst-case optimality.

The algorithm to compute the Minkowski sum of two convex polytopes can be used to compute the Minkowski sum of multiple convex polytopes. We present the algorithm and analyze its complexity in the next section.

5 The Minkowski Sum of Multiple Convex Polytopes

Let $P_1, P_2, \dots, P_r$ be r convex polytopes of the face lattice sizes $n_1, n_2, \dots, n_r$, respectively. We can compute the Minkowski sum $\bigoplus_{i=1}^r P_i$ in $O(d^\omega \sum_{j=1}^r \prod_{i=1}^j n_i) = O(d^\omega \cdot r \cdot \prod_{i=1}^r n_i)$ in the worst case by the method presented in Sect. 4 after sorting the polytopes in ascending order of the sizes of face lattices. The worst-case complexity of the resulting algorithm is tight for $r < d$. For $r \geq d$, however, it is known that the worst-case space complexity of the face lattice of the resulting polytope does not increase with r as an exponent [21]. Thus the time complexity has an upper bound. We describe this bound from the citation for reference. Let $M = O(\sum_{S \in \mathcal{C}} |\bigoplus_{P \in S} P|)$ where $\mathcal{C}$ is the set of all $d - 1$ combinations of r polytopes. By Stirling's approximation, this quantity is asymptotically $\approx (er/(d - 1))^{d-1}$ times maximum size of $(d - 1)$ -subset Minkowski sum among r polytopes. The size of the Minkowski sum for any $r, r \geq d$ is bounded by M . Consequently, we give the running time for the computation of the Minkowski sum of r convex polytopes in the theorem below.

Theorem 3. *The Minkowski sum $\bigoplus_{i=1}^r P_i$ of r convex polytopes P_i 's of face lattice sizes n_i 's, $1 \leq i \leq r$, can be computed in time $O(d^\omega \min\{NM, r \prod_{i=1}^r n_i\})$, where N is the total input size $\sum_{i=1}^r n_i$ and M is the face lattice size of the resultant Minkowski sum. The space complexity of the algorithm is $O(d^2 M)$. Moreover, the algorithm is worst-case optimal for fixed $d > 3$ and $r < d$.*

Proof. The proof follows from the observation that the size of the face lattice of the sum is always larger than the size of the face lattice of any of the

summands. The time complexity for $r \geq d$ is $O(\sum_{i=1}^r d^\omega n_i M) = O(d^\omega NM)$ by the application of Theorem 2. For $r < d$ too, the expression for time-complexity, $O(d^\omega r \prod_{i=1}^r n_i)$, which might be larger than the tighter complexity of $O(d^\omega \sum_{i=1}^r n_i M)$, follows directly from Theorem 2. Thus the time-complexity is $O(d^\omega \min\{NM, r \prod_{i=1}^r n_i\})$. The correctness of the algorithm follows from the correctness of the algorithm of previous section. The space requirement is due to augmented face lattice of the output polytope in the intermediate steps. The argument for worst-case optimality for fixed d is same as in the proof of Theorem 2. $\square$

6 Concluding Remarks and Acknowledgments

In the paper, we presented a worst-case optimal algorithm to compute the Minkowski sum with better bounds than the currently known algorithms. We acknowledge anonymous referees for providing constructive suggestions and suggesting improvements in the paper.

References

1. Adiprasito, K.A., Sanyal, R.: Relative Stanley-Reisner theory and upper bound theorems for Minkowski sums. *Publications mathématiques de l'IHÉS* **124**(1), 99–163 (2016)
2. Agarwal, P.K., Flato, E., Halperin, D.: Polygon decomposition for efficient construction of Minkowski sums. *Comput. Geom.* **21**(1–2), 39–61 (2002)
3. Bekker, H., Roerdink, J.B.T.M.: An efficient algorithm to calculate the Minkowski sum of convex 3D polyhedra. In: Alexandrov, V.N., Dongarra, J.J., Juliano, B.A., Renner, R.S., Tan, C.J.K. (eds.) *ICCS 2001*. LNCS, vol. 2073, pp. 619–628. Springer, Heidelberg (2001). https://doi.org/10.1007/3-540-45545-0_71
4. de Berg, M., Cheong, O., van Kreveld, M., Overmars, M.: *Computational Geometry: Algorithms and Applications*, 3rd edn. Springer, Heidelberg (2008). <https://doi.org/10.1007/978-3-540-77974-2>
5. Chan, T.M.: Output-sensitive results on convex hulls, extreme points, and related problems. *Discret. Comput. Geom.* **16**(4), 369–387 (1996). <https://doi.org/10.1007/BF02712874>
6. Chazelle, B.: An optimal convex hull algorithm in any fixed dimension. *Discret. Comput. Geom.* **10**(4), 377–409 (1993). <https://doi.org/10.1007/BF02573985>
7. Chazelle, B.M.: Convex decompositions of polyhedra. In: *Proceedings STOC*, pp. 70–79 (1981)
8. Chazelle, B.M., Dobkin, D.P.: Optimal convex decompositions. In: Toussaint, G.T. (ed.) *Computational Geometry. Machine Intelligence and Pattern Recognition*, vol. 2, pp. 63–133 (1985)
9. Chew, L.P., Scot Drysdale, R.L.: Voronoi diagrams based on convex distance functions. In: *Proceedings SoCG*, pp. 235–244 (1985)
10. Das, S., Nandy, A., Sarvottamananda, S.: Radius, diameter, incenter, circumcenter, width and minimum enclosing cylinder for some polyhedral distance functions. *Discret. Appl. Math.* (2020, in press)

11. Edelsbrunner, H.: Algorithms in Combinatorial Geometry. EATCS Monographs on Theoretical Computer Science. Springer, Heidelberg (1987). <https://doi.org/10.1007/978-3-642-61568-9>
12. Fogel, E., Halperin, D.: Exact Minkowski sums of convex polyhedra. In: Proceedings SoCG, pp. 382–383 (2005)
13. Fukuda, K.: From the zonotope construction to the Minkowski addition of convex polytopes. *J. Symb. Comput.* **38**(4), 1261–1272 (2004)
14. Fukuda, K., Weibel, C.: Computing all faces of the Minkowski sum of V-polytopes. In: Proceedings of the 17th CCCG, pp. 253–256 (2005)
15. Gritzmann, P., Sturmfels, B.: Minkowski addition of polytopes: computational complexity and applications to Gröbner basis. *SIAM J. Discret. Math.* **6**(2), 246–269 (1993)
16. Grünbaum, B., Kaibel, V., Klee, V., Ziegler, G.M.: Convex Polytopes. Graduate Texts in Mathematics. Springer, Heidelberg (2003). <https://doi.org/10.1007/978-1-4613-0019-9>
17. Karavelas, M.I., Tzanaki, E.: The maximum number of faces of the Minkowski sum of two convex polytopes. In: Proceedings SODA, pp. 11–28 (2012)
18. Le Gall, F.: Powers of tensors and fast matrix multiplication. In: Proceedings ISSAC, pp. 296–303 (2014)
19. Ramkumar, G.D.: An algorithm to compute the Minkowski sum outer-face of two simple polygons. In: Proceedings SoCG, pp. 234–241 (1996)
20. Seidel, R.: Constructing higher-dimensional convex hulls at logarithmic cost per face. In: Proceedings STOC, pp. 404–413 (1986)
21. Weibel, C.: Maximal F-vectors of Minkowski sums of large numbers of polytopes. *Discret. Comput. Geom.* **47**(3), 519–537 (2012)
22. Wein, R.: Exact and efficient construction of planar Minkowski sums using the convolution method. In: Azar, Y., Erlebach, T. (eds.) ESA 2006. LNCS, vol. 4168, pp. 829–840. Springer, Heidelberg (2006). https://doi.org/10.1007/11841036_73